

centric networking, which requires higher security to be enforced in the network transmission along with a policy driven security framework using blockchain principles.

It is widely used in the entertainment industry for media transmission and streaming services, including workflow processing in media encoding, approval and stream processing using a cloud based large storage for media storage, and encoding and transmission across the media broadcasting service.

Last year, KardiaChain, which is a decentralised interoperable and self-optimised blockchain system, announced it was using BlockCloud in its platform for data sharing and interchain communication in government projects like Internet of Vehicles (IoV), smart healthcare and smart homes.

With this joint partnership, blockchain use cases can be expected

to reach new heights. As an example, a payment channel for mobile payments can be set up across a country, reducing debit/credit card based transactions by using decentralised blockchain based payment transactions.

To define the security policies and business logic for decentralised communication, KardiaChain uses KardiaChain smart contract language

(KSCL). It can be used to develop cross-chain smart contracts easily to process transactions between Kardia ledgers seamlessly without compromising security standards. It also uses SCN communication for better network transmission, even when the user load increases, by using IoT edge nodes and private edge nodes for service interoperability. **END** 

References

- [1] <https://www.block-cloud.io/>
- [2] <https://www.block-cloud.io/blockcloudtechnicalwhitepaper.pdf>
- [3] CloudPos: A Proof of Stake Consensus Design for Blockchain Integrated Cloud: <https://ieeexplore.ieee.org/document/8457813>

By: Dr Anand Nayyar and Dr Magesh Kasthuri

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam. He loves to explore open source technologies, IoT, cloud computing, deep learning and cybersecurity.

Dr Magesh Kasthuri is a senior distinguished member of the technical staff and principal consultant at Wipro Ltd. This article expresses his view and not that of Wipro.

Continued from page 59...

```
Fri Aug 28 16:59:12 2020
: Node ID: 4426
Fri Aug 28 17:00:07 2020
: In command handler -> join -> sending msg to ip 127.0.0.1 port 9091
Fri Aug 28 17:00:07 2020
: Recieved RoutingUpdate request
Fri Aug 28 17:00:07 2020
: Handling Routing Update request from IP: 127.0.0.1 Port: 9091 NodeID: 7353
Fri Aug 28 17:00:07 2020
: adding to right leaf set IP : 127.0.0.1 Port : 9091 NodeID: 7353
Fri Aug 28 17:00:07 2020
: adding to neighbour set IP : 127.0.0.1 Port : 9091 NodeID: 7353
Fri Aug 28 17:00:07 2020
: adding to routing table IP : 127.0.0.1 Port : 9091 NodeID: 7353 Prefix,index = 0,7
Fri Aug 28 17:00:07 2020
: In incrementRecievedUpdateCount
Fri Aug 28 17:00:07 2020
: before increment: 0
Fri Aug 28 17:00:07 2020
: after increment: 1
Fri Aug 28 17:00:07 2020
: adding to routing table IP : 127.0.0.1 Port : 9093 NodeID: 0072 Prefix,index = 0,0
Fri Aug 28 17:00:07 2020
: Node ID: 7353
Fri Aug 28 17:00:02 2020
: Recieved JoinMe request for IP: 127.0.0.1 Port: 9093
Fri Aug 28 17:00:02 2020
: Handling JoinMe request for IP: 127.0.0.1 Port: 9093 NodeID: 0072
Fri Aug 28 17:00:02 2020
: Next best route IP: 127.0.0.1 Port: 9091 NodeID: 7353
Fri Aug 28 17:00:02 2020
: Sending RoutingUpdate packet to IP: 127.0.0.1 Port: 9093
Fri Aug 28 17:00:02 2020
: Error - PeerHandler::handleRpc() Exception received: connection , fd: 5 For fd: 5
Fri Aug 28 17:00:02 2020
: Recieved AllStateUpdate request
Fri Aug 28 17:00:02 2020
: Handling ALL State Update request from IP: 127.0.0.1 Port: 9093 NodeID: 0072
Fri Aug 28 17:00:02 2020
: Recieved AllStateUpdate request
Fri Aug 28 17:00:02 2020
: adding to left leaf set IP : 127.0.0.1 Port : 9093 NodeID: 0072
Fri Aug 28 17:00:02 2020
: Handling ALL State Update request from IP: 127.0.0.1 Port: 9093 NodeID: 0072
Fri Aug 28 17:00:02 2020
: Node ID: 7353
```

Figure 8: Logs from Node1 and Node2

This implementation is based on C++. We used protocol buffers for communication and UNIX provided interfaces for accessing system resources. This design can be used as a basis for developing applications like messenger, publish-subscribe system where the publisher stores the list of subscribers and can efficiently route the message to subscribers, and global file sharing systems. **END** 

References

- [1] 'Pastry: Scalable, decentralised object location and routing for large-scale peer-to-peer Systems' by Antony Rowstron and Peter Druschel, available at <http://rowstron.azurewebsites.net/PAST/pastry.pdf>
- [2] 'CAP_theorem' by Eric Brewer, available at https://en.wikipedia.org/wiki/CAP_theorem
- [3] 'All Things Distributed' by Werner Vogels, available at http://www.allthingsdistributed.com/2007/10/amazons_dynamo.html

By: Gaurav Arora and B. Thangaraju

The authors are associated with the open source technology lab in the International Institute of Information Technology, Bengaluru.